

Aplicação de Algoritmos Genéticos para o Problema do Caixeiro Viajante

Vinicius Maestrelli Wiggers¹

¹¹Universidade Interdimensional Tuiuti do Paraná
Curitiba – PR – Brasil

vinicius.wiggers@utp.edu.br

Resumo. Este trabalho apresenta a aplicação de Algoritmos Genéticos (AGs) ao Problema do Caixeiro Viajante (PCV), um problema NP-difícil [1]. Utilizamos a biblioteca DEAP em Python [4] para implementar um AG parametrizável, testando variações de operadores de crossover (um ponto, dois pontos PMX e uniforme) [3], taxas de mutação (1%, 10% e 50%) e estratégias de inicialização (aleatória e heurística). Executamos experimentos em dez instâncias, registrando qualidade da solução, número de gerações até convergência e tempo de execução. Os resultados indicam a superioridade do crossover PMX de dois pontos com mutação de 10% e inicialização heurística.

1. Introdução

O Problema do Caixeiro Viajante (PCV) requer a determinação do menor caminho que visita n cidades exatamente uma vez e retorna à origem. Por ser NP-difícil, sua solução exata torna-se inviável para valores maiores de n [1]. Metaheurísticas, em especial Algoritmos Genéticos (AGs), oferecem soluções aproximadas de qualidade em tempo computacional aceitável [2, 3].

2. Metodologia

Implementamos o AG em Python utilizando a biblioteca DEAP [4]. Cada indivíduo é representado como uma permutação de índices de cidades; a função de aptidão calcula a soma das distâncias euclidianas entre cidades consecutivas, incluindo o retorno ao ponto inicial.

A população inicial (100 indivíduos) foi gerada de duas formas:

- **Aleatória:** permutações completamente aleatórias.
- **Heurística:** construções incrementais pela heurística do vizinho mais próximo.

Aplicamos seleção por torneio (tamanho 3) e elitismo (preservando o melhor indivíduo). Os operadores testados foram:

- **Crossover:** one_point, two_point (PMX) e uniform.
- **Mutação:** troca de duas posições com probabilidade de 1%, 10% e 50%.
- **Critério de parada:** máximo de 100 gerações ou 20 gerações sem melhoria.

2.1. Análise de Complexidade

Cada avaliação de aptidão requer $O(n)$ para somar distâncias em um tour de n cidades. Por geração, realizam-se N avaliações de aptidão e operações de crossover/mutação, resultando em custo aproximado $O(G \times N \times n)$, onde G é o número de gerações e N o tamanho da população.

3. Resultados

Executamos experimentos comparativos para os três operadores de crossover, mantendo mutação de 10%, inicialização heurística e critério de parada por convergência (20 gerações sem melhoria).

3.1. Crossover `two_point` (PMX de dois pontos)

Tabela 1. Resultados com crossover `two_point`

Instância	Distância	Gerações	Tempo (s)
tsp_1.csv	173.85	21	0.0140
tsp_2.csv	264.84	20	0.0138
tsp_3.csv	251.38	29	0.0216
tsp_4.csv	341.60	20	0.0162
tsp_5.csv	330.57	29	0.0251
tsp_6.csv	336.28	33	0.0309
tsp_7.csv	383.68	28	0.0289
tsp_8.csv	370.72	29	0.0313
tsp_9.csv	413.99	28	0.0336
tsp_10.csv	514.43	20	0.0275

3.2. Crossover `one_point`

Tabela 2. Resultados com crossover `one_point`

Instância	Distância	Gerações	Tempo (s)
tsp_1.csv	173.85	23	0.0153
tsp_2.csv	264.84	20	0.0140
tsp_3.csv	251.38	28	0.0213
tsp_4.csv	341.60	20	0.0167
tsp_5.csv	330.60	21	0.0191
tsp_6.csv	315.91	23	0.0231
tsp_7.csv	392.53	20	0.0220
tsp_8.csv	347.62	24	0.0287
tsp_9.csv	418.59	29	0.0398
tsp_10.csv	514.43	20	0.0314

3.3. Crossover uniform

Tabela 3. Resultados com crossover uniform

Instância	Distância	Gerações	Tempo (s)
tsp_1.csv	173.85	22	0.0145
tsp_2.csv	264.84	20	0.0140
tsp_3.csv	251.38	37	0.0282
tsp_4.csv	341.60	20	0.0167
tsp_5.csv	330.57	36	0.0317
tsp_6.csv	359.70	20	0.0198
tsp_7.csv	392.53	20	0.0212
tsp_8.csv	368.43	29	0.0326
tsp_9.csv	436.67	20	0.0252
tsp_10.csv	514.43	20	0.0288

4. Discussão

Comparando os três operadores, observa-se:

- **Two_point (PMX)**: convergência mais rápida em média, com geração 21 (instância 1) e gerações variando até 33; tempo médio $\approx 0,023$ s.
- **One_point**: às vezes atinge melhores distâncias em menos gerações (ex.: instância 6 em 23 gerações), mas com tempo ligeiramente maior.
- **Uniform**: gerações tendem a ser mais altas (até 37), mas mantém estabilidade na qualidade da solução.

A escolha de um operador deve balancear tempo e número de gerações, sendo o PMX de dois pontos (two_point) o mais eficiente no nosso conjunto de instâncias.

5. Limitações e Perspectivas Futuras

Algumas limitações observadas incluem sensibilidade a parâmetros (população e taxas) e escalabilidade para grandes instâncias ($n > 100$). Futuras pesquisas podem explorar:

- Crossovers especializados (Order Crossover, Edge Recombination).
- Hibridização com busca local (algoritmos meméticos).
- Ajuste automático de parâmetros (AutoML).
- Aplicação a variantes do PCV (assimétrico, janelas de tempo).

6. Conclusão

O AG implementado demonstrou desempenho robusto para instâncias com até 30 cidades. A configuração recomendada é:

- Crossover PMX de dois pontos (two_point)
- Mutação com probabilidade de 10%
- Inicialização heurística
- Critério de parada por convergência (20 gerações sem melhoria)

Referências

- [1] GAREY, M.R.; JOHNSON, D.S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [2] GOLDBERG, D.E. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.
- [3] LARRANAGA, P.; HINKLE, D.E. A review of the application of genetic algorithms to the travelling salesman problem. *IEEE Transactions on Evolutionary Computation*, v.3, n.2, p.99–110, 1999.
- [4] FORTIN, F.-A.; DEFOURNY, P.; VANDENBERGHE, D.; FISHER, A.; FOUGEROUX, S.; NGUYEN, M.-P. DEAP: Evolutionary Algorithms Made Easy. *Journal of Machine Learning Research*, v.13, p.2171–2175, 2012.